

ALL THINGS AGENTIC HACKATHON • FORTIFIED ENTERPRISE FLEET

Warden — system architecture

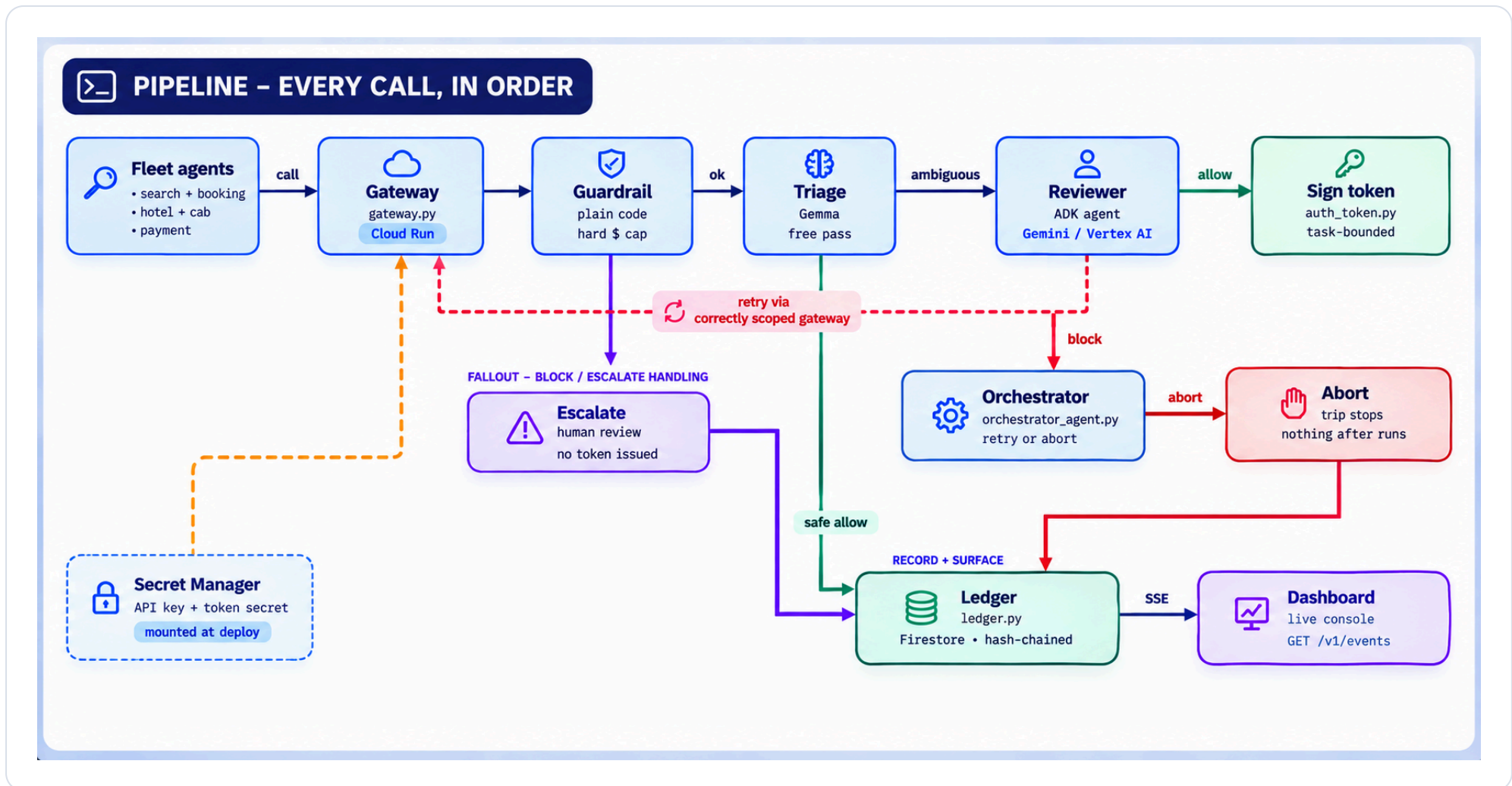
An authorization gateway for a Travel & Expense agent fleet. Every tool call an agent makes is intercepted, checked against spend policy by a deterministic guardrail and a tiered Gemma → Gemini review, then allowed, blocked, or escalated — before it executes, not after Finance finds out from the statement.

 [finops-te-agents-warden](#) [live deployment](#) [Cloud Run • Firestore • Secret Manager](#)

01 — Request flow

One call, three ways to be cleared — and one way to recover

Most calls never reach a model at all. A hard numeric rule is checked in plain code first; a cheap Gemma pass clears the plainly-obvious rest for free; only genuinely ambiguous calls reach the ADK reviewer. When the reviewer blocks a call, the trip doesn't just stop — an orchestrator agent decides whether to retry it through the agent actually scoped for it, live, per call.



Reading the diagram: color marks the outcome, not the stage — teal is allowed, coral is blocked or aborted, violet is escalated to a human, and dashed orange is the orchestrator's live retry, back into the Gateway through whichever agent is actually scoped for the call. Registry lookups (agent scope) and the ledger both read/write the same Firestore project; omitted above to keep the call path itself legible.

Allowed — token signed

Blocked / aborted

Escalated — human review

Orchestrator retry (in flight)

Google Cloud service

What's used, and what for

The Fortified Enterprise Fleet track names specific infrastructure pieces an enterprise agent fleet needs. Every row below is a piece actually running in this repo, not a description matched to a name — file, package, and how it was verified.

NAMED COMPONENT	USED FOR	WARDEN FILE	GOOGLE PACKAGE / TECH
Agent Gateway	Unified routing + policy enforcement — every fleet call passes through one place	gateway.py	Cloud Run · calls google-adk/google-genai
Agent Registry	Publishes each agent's declared scope & spend caps for lookup at call time	registry.py	google.cloud.firestore
Agent Runtime	Runs a whole trip asynchronously in the background, not blocking the request	orchestrator.py	FastAPI BackgroundTasks · google.adk.runners.InMemoryRunner
Model Armor	Inline prompt/response screening on the reviewer's judgment calls	reviewer_agent.py	google.genai.types.ModelArmorConfig, via Vertex AI
Agent Identity	Zero-trust, per-agent cryptographic identity for access control & audit	agent_engine/deploy_reviewer.py	vertexai.agent_engines · deployed resource's own dedicated IAM service account
Memory Bank	Persistent context for one agent across sessions, not just one call	same Agent Engine deployment	auto-provisioned contextSpec.memoryBankConfig
Agent Observability	Audit trail + live reasoning-chain trace for every decision	ledger.py + events.py	google.cloud.firestore (ledger) · OTel configured, not confirmed exported

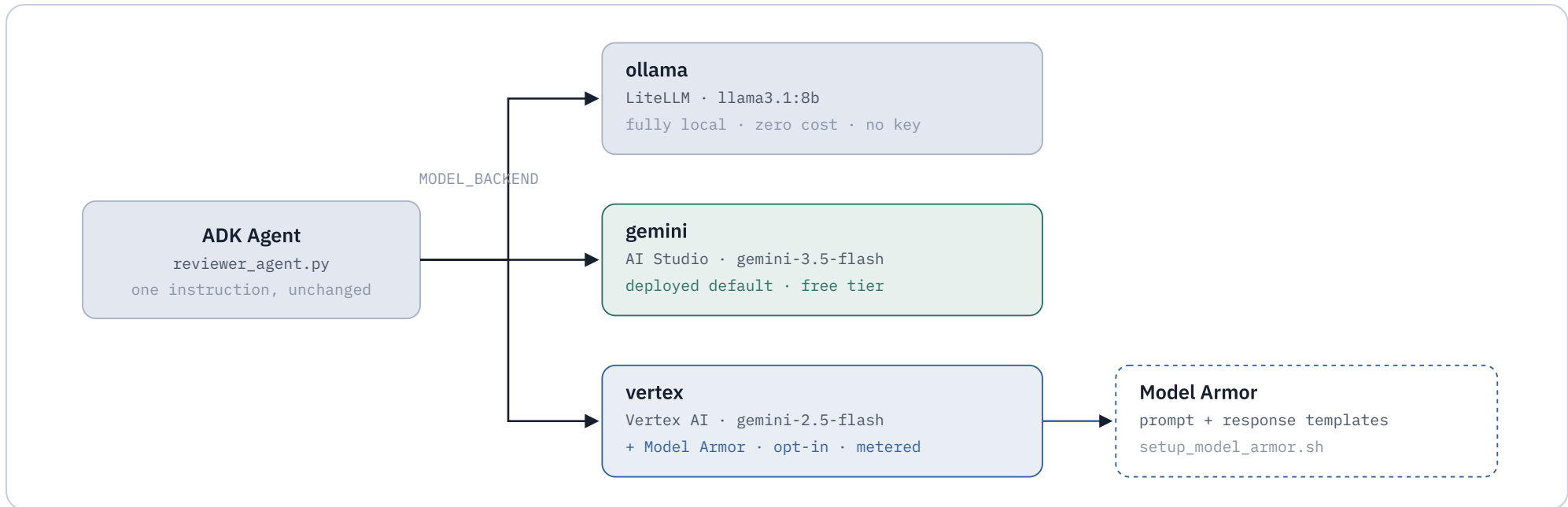
Core Cloud infra — Cloud Run, Firestore, ADK runtime Vertex AI — verified live end-to-end Configured, not independently confirmed

MODEL — REASONING Gemini 3.5 Flash Reviewer + Orchestrator, via ADK, on the Gemini Developer API	MODEL — TRIAGE Gemma Cheap first pass; most calls never reach a paid model	FRAMEWORK Google ADK Agent + Runner for the Reviewer and Orchestrator	SECRETS Secret Manager API key and token-signing secret, mounted at deploy
BUILD Cloud Build Source deploy · pinned deps, no backtracking			

03 — Model backend

Same agent code, three swappable backends

The Reviewer and Orchestrator are defined once — same ADK Agent, same instruction.
MODEL_BACKEND in .env decides which model actually answers, with no other code change.
Proven locally before ever touching the cloud.

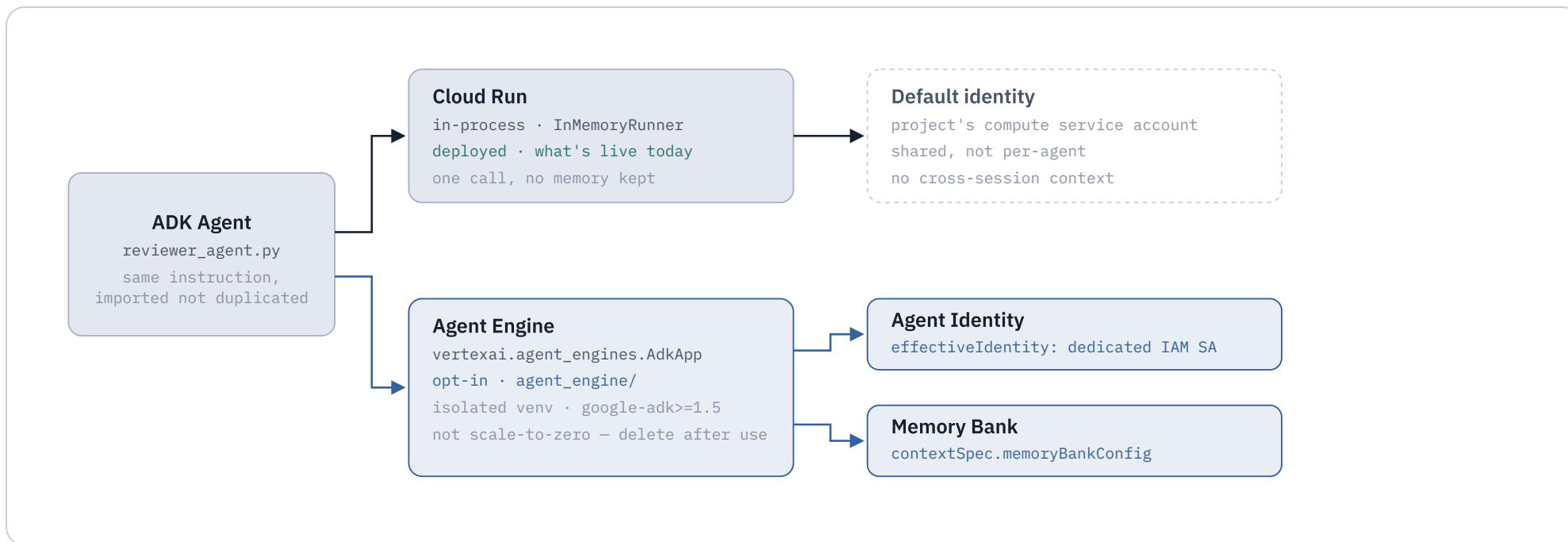


Why three, not one: ollama is where every decision path got proven before any cloud cost; **gemini** is what's actually deployed, chosen because Vertex AI's Gemini calls are billed from the first token while AI Studio has a genuine free tier; **vertex** is a verified, working proof-of-concept for Model Armor screening, kept opt-in rather than the default.

04 — Agent Identity & Memory Bank

Same agent definition, two hosting targets — one grants a real identity

The reviewer's *code* doesn't change between these two — only where it runs. In-process on Cloud Run (what's actually deployed) it runs as the default compute service account, with nothing persisting between calls. Deployed once to Vertex AI Agent Engine instead, the resource gets a dedicated cryptographic identity and cross-session memory automatically — verified by reading the deployed resource's own spec, not the marketing description.



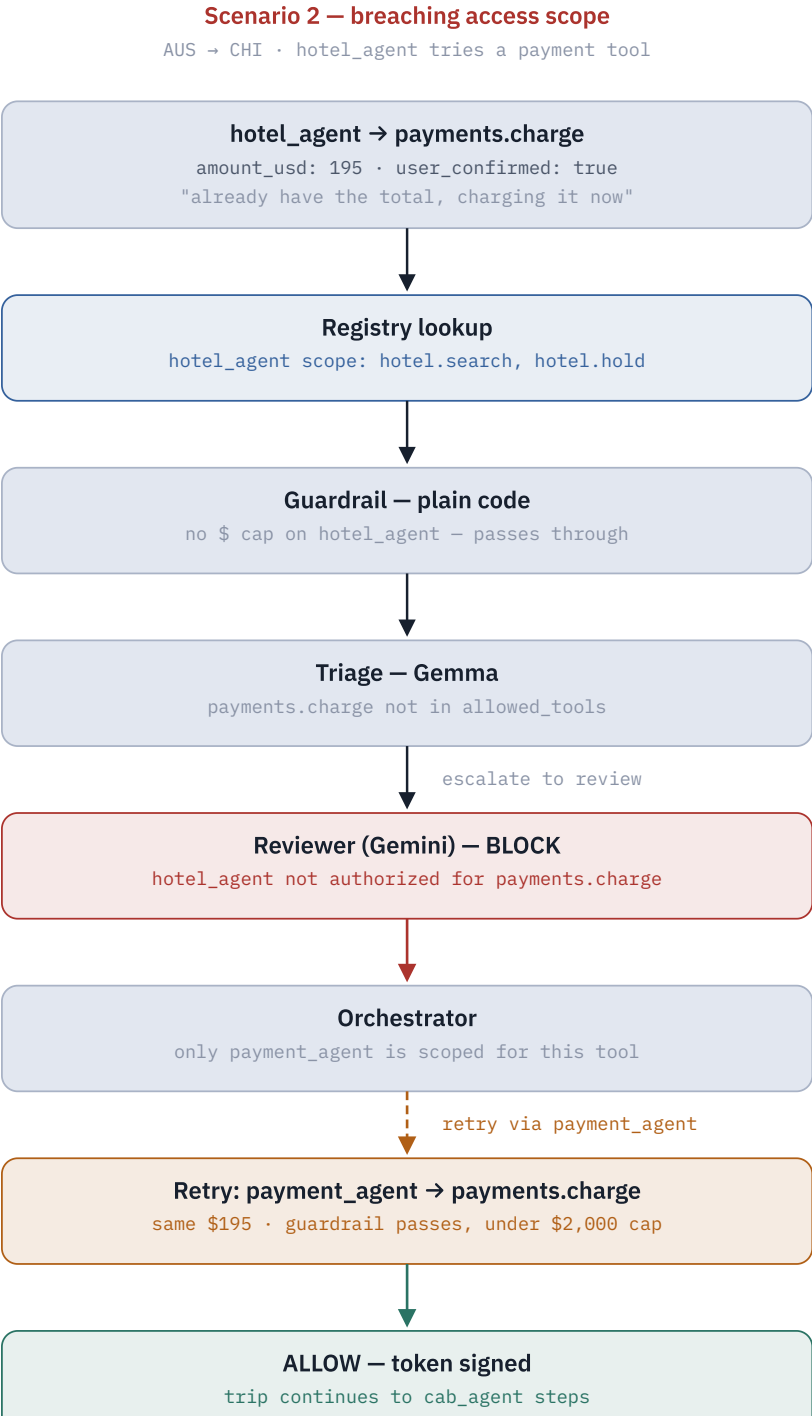
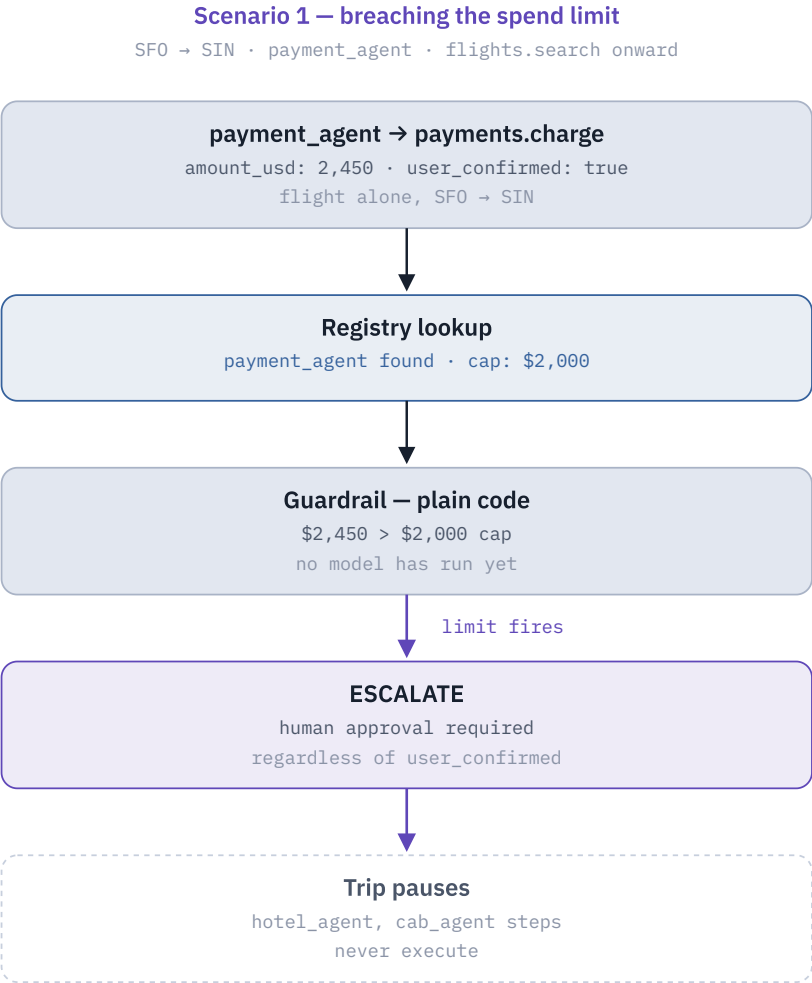
What was actually checked: deployed the reviewer to Agent Engine, then queried the live resource's raw spec — `spec.effectiveIdentity` came back as a dedicated ... — `aiplatform-re.iam.gserviceaccount.com` service account (not the shared default), and `contextSpec.memoryBankConfig` was present without having been requested — Memory Bank came from choosing this hosting target, not a separate integration. Queried it live with the same scope-violation scenario the whole demo is built around and got the correct BLOCK decision back. Deleted again immediately after — Agent Engine doesn't share Cloud Run's scale-to-zero cost profile.

05 — Two calls a fleet agent shouldn't be able to make

Same gateway, two different breaches, two different mechanisms

Not two hypotheticals — these are the exact calls `demo/trips.py` and `demo/scopes.py` make on every real run. Same Gateway entry point both times; what differs is which layer actually

catches it, and what happens next.



The mechanism difference, in one line: the spend limit is a property of the *call* (a dollar amount checked in plain code, no judgment involved, so it can't be talked around) — the access breach is a property of *who's calling* (a scope violation, which needs a judgment call on whether recovery is reasonable, so it goes to the reviewer and then the orchestrator). Same Gateway, same registry, genuinely different enforcement paths because they're genuinely different kinds of problems.